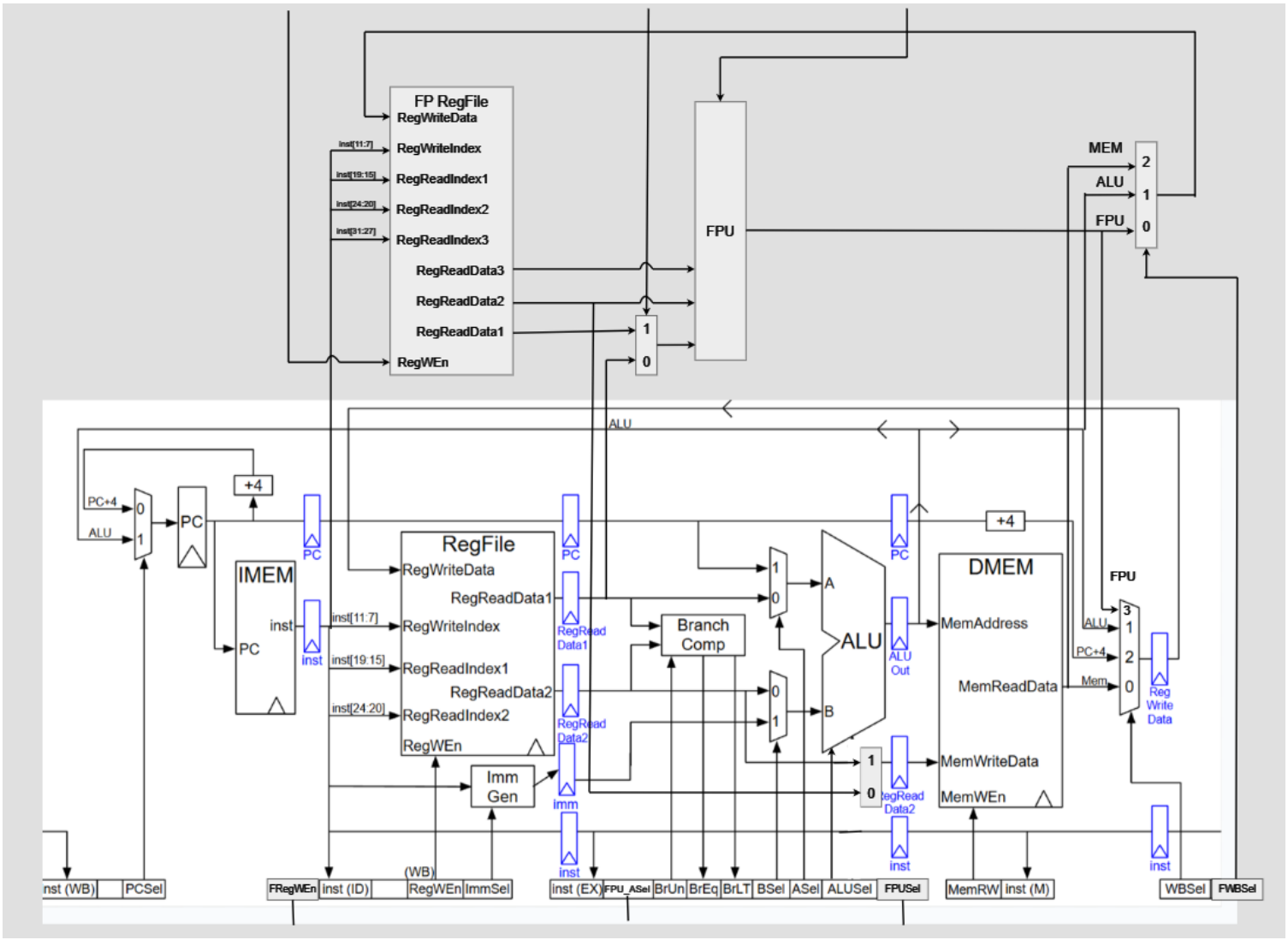
Lab 1 Open-Ended Portion Report: Custom RV32IMF Implementation

1. Project Overview

In this open-ended portion of Lab 1, our team undertook a custom project to significantly extend the Sodor processor capabilities. We moved beyond the standard requirements to implement a full hardware Floating Point Unit (FPU) compliant with the RV32F single-precision floating-point extension, alongside the RV32M integer multiplication extension. This project involved comprehensive modifications to the datapath, control logic, and register file structures to support hardware-accelerated floating-point arithmetic, fused multiply-add operations, and mixed integer-floating point interactions.

2. Implementation Details



2.1 RV32M Extension

Hardware implementation of the standard integer multiplication and division instructions: lower-bits multiplication (`mul`), high-bits signed multiplication (`mulh`), high-bits signed-unsigned multiplication (`mulhsu`), high-bits unsigned multiplication (`mulhu`), signed division (`div`), unsigned division (`divu`), signed remainder (`rem`), and unsigned remainder (`remu`). This involves:

1. Arithmetic Logic Unit (ALU) Extension for new operations
2. Control Path Extension for new instructions

2.2 RV32F FLW & FSW

Hardware implementation of Floating-point Load Word (FLW) and Floating-point Store Word (FSW) instructions for the RV32F extension, involving:

1. Floating Point (FP) RegFile implementation
2. FP RegFile input & output paths
3. FP RegFile and Data Memory (DMEM) interface
4. Control Path Extension for new instructions

2.3 RV32F R-type Instructions

Hardware implementation of R-type instructions for the RV32F extension, involving:

1. FPU implementation

2. FPU input & output paths
3. Bypass and stall Logic for FPU operands
4. Control Path Extension for new instructions

Note: fdiv.s and fsqrt.s are not implemented as their hardware implementation are multi-cycle and require large modifications.

2.4 RV32F R4-type (FMA) Instructions

Hardware implementation of R4-type (Fused Multiply-Add, FMA) instructions for the RV32F extension, involving:

1. FPU Extension for 3-input operations
2. FP RegFile Extension for 3-register access
3. Bypass and stall Logic for additional FPU operand
4. Control Path Extension for new instructions

2.5 RV32F Int-FP Conversion Instructions

Hardware implementation of Integer-Floating Point (Int-FP) conversion instructions (fcvt and fmv) for the RV32F extension, involving:

1. FPU Extension for conversion operations
2. FPU input mux for possible integer input
3. Append FPU writeback to integer regfile
4. Int-FP and FP-Int bypass paths
5. Control Path Extension for new instructions

2.6 RV32F FCSR

Hardware implementation of Floating-point Control and Status Register (FCSR) for the RV32F extension, involving:

1. Floating-Point Exception Flags (FFLAGS) and Floating-Point Rounding Mode (FRM) implementation
2. FCSR read and write data paths
3. FRM rounding mode control logic
4. FPU FFLAGS propagation logic

3. Experimental Evaluation

3.1 Objective and Methodology

The benchmark utilized is `mix.c`, located in `generators/riscv-sodor/test/custom-bmarks/`, performing floating-point matrix multiplication. Three distinct binaries were compiled to compare Instruction Set Architecture (ISA) extensions:

- `mix_i.riscv`: Targetting the RV32I ISA.
- `mix_im.riscv`: Targetting the RV32IM ISA; includes hardware integer multipliers but still relies on software for floating-point operations.
- `mix_imf.riscv`: Targetting the RV32IMF ISA; directly utilizes hardware FPU instructions and dedicated floating-point registers.

3.2 Summary of Experimental Results

Simulations were conducted for three matrix dimensions (4×4, 8×8, 16×16) using the Verilator simulator. The results extracted via `tracer.py` are summarized below:

Scale	Binary Version	CPI	Total Cycles	Instructions	Branch %	FP %
4×4	mix_i / mix_im	1.256	936	745	13.557%	0.000%
4×4	**mix_imf**	1.302	936	719	11.266%	4.451%
8×8	mix_i / mix_im	1.198	140,256	117,062	14.901%	0.000%
8×8	**mix_imf**	1.154	9,846	8,531	8.639%	10.128%
16×16	mix_i / mix_im	1.190	1,056,366	887,636	14.630%	0.000%
16×16	**mix_imf**	**1.113**	**86,796**	**77,979**	**6.022%**	**6.935%**

(Refer to Figure 1, Figure 2, and Figure 3 in Appendix for comparative charts)

3.3 Detailed Analysis

3.3.1 RV32I vs. RV32IM

Data shows that `mix_i` and `mix_im` perform identically across all scales. This confirms that the software floating-point emulation library relies solely on basic integer arithmetic and does not utilize hardware integer multipliers. For floating-point intensive tasks, the M-extension provides no lateral performance benefit.

3.3.2 Scalability

- **Startup Overhead (4×4):** Total cycles are identical (936 vs. 936), so the measured speedup is **1.0x**. Instruction count drops only from 745 to 719 (−26, about **3.5%**), which is too small to overcome fixed startup cost. The Cycles Per Instruction (CPI) difference (1.256 vs. 1.302) confirms the run is dominated by non-kernel overhead rather than FPU throughput.
- **Mid-Scale (8×8):** Total cycles fall from 140,256 to 9,846, yielding about **14.25x** speedup. Instruction count drops from 117,062 to 8,531 (−92.7%), indicating the hardware FPU removes most software emulation work.
- **Steady State (16×16):** Total cycles drop from 1,056,366 to 86,796, a **12.17x** speedup. Instruction count decreases from 887,636 to 77,979 (**−91.2%**), while CPI also improves (1.190 → 1.113), showing both fewer instructions and better pipeline efficiency at scale.

3.3.3 Branch Pressure and Pipeline Efficiency

At 16×16, Branch% falls from **14.63%** to **6.02%**, a **58.9%** relative reduction. This drop aligns with the removal of software FP control-flow (sign/exp/round checks). Fewer branches mean fewer pipeline flushes and a steadier fetch stream, which contributes to the lower CPI and higher effective throughput seen at large scales.

3.3.4 CPI Evolution: Read-After-Write (RAW) Hazards vs. Bypass Logic

At 4×4, CPI is slightly worse for IMF (1.302 vs. 1.256), which is consistent with short, dependency-heavy sequences where RAW hazards dominate and fixed overhead masks benefits. As the workload scales, CPI steadily improves to **1.113** at 16×16, outperforming Soft-FP (1.190). This indicates the Triple-Read-Port Register File and Bypass Logic keep the FP pipeline fed in steady state, reducing stall cycles that are common in software-emulated FP with stack traffic.

Appendix: Experimental Figures

Figure 1: Total Cycles by Scale

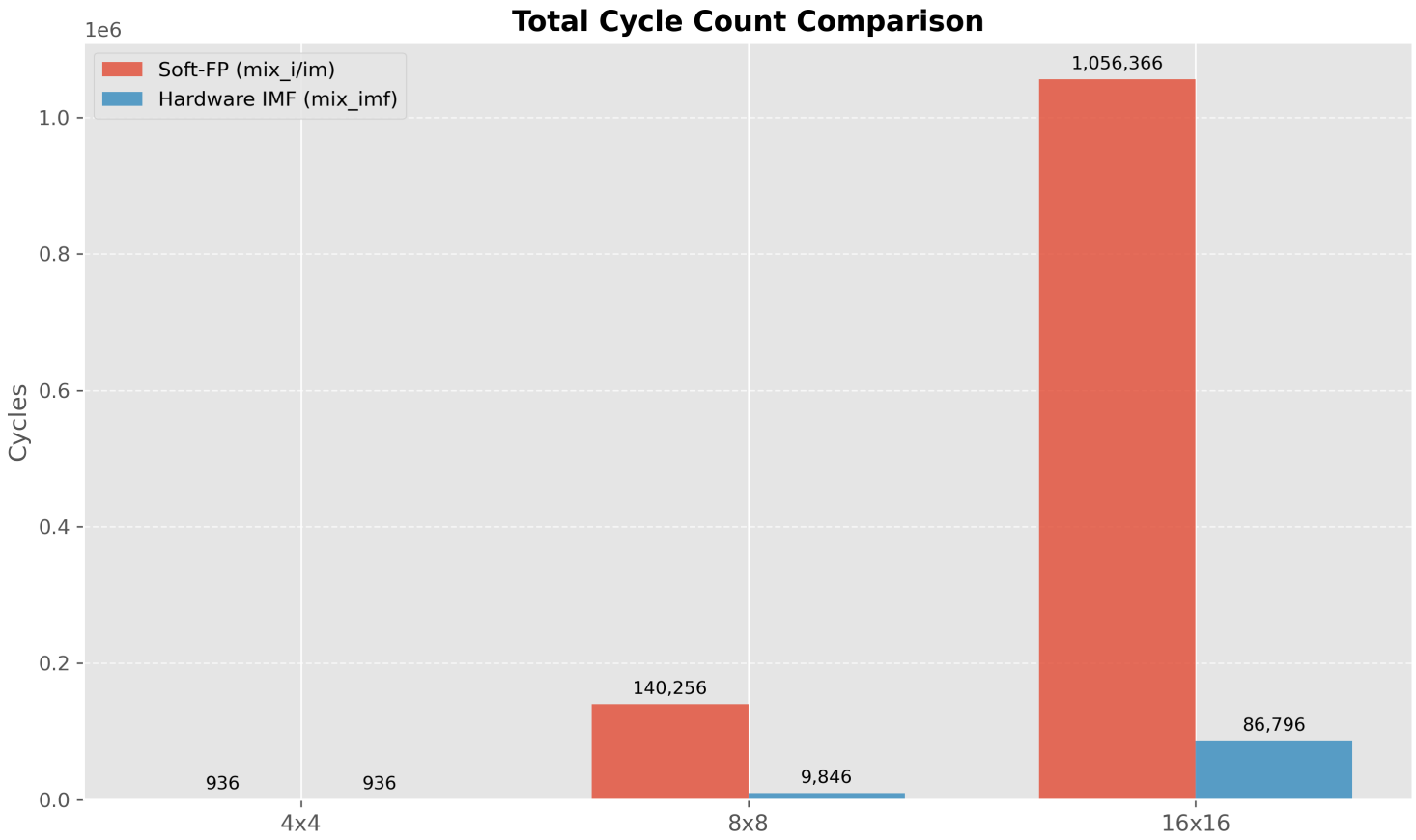


Figure 2: Instruction Count by Scale

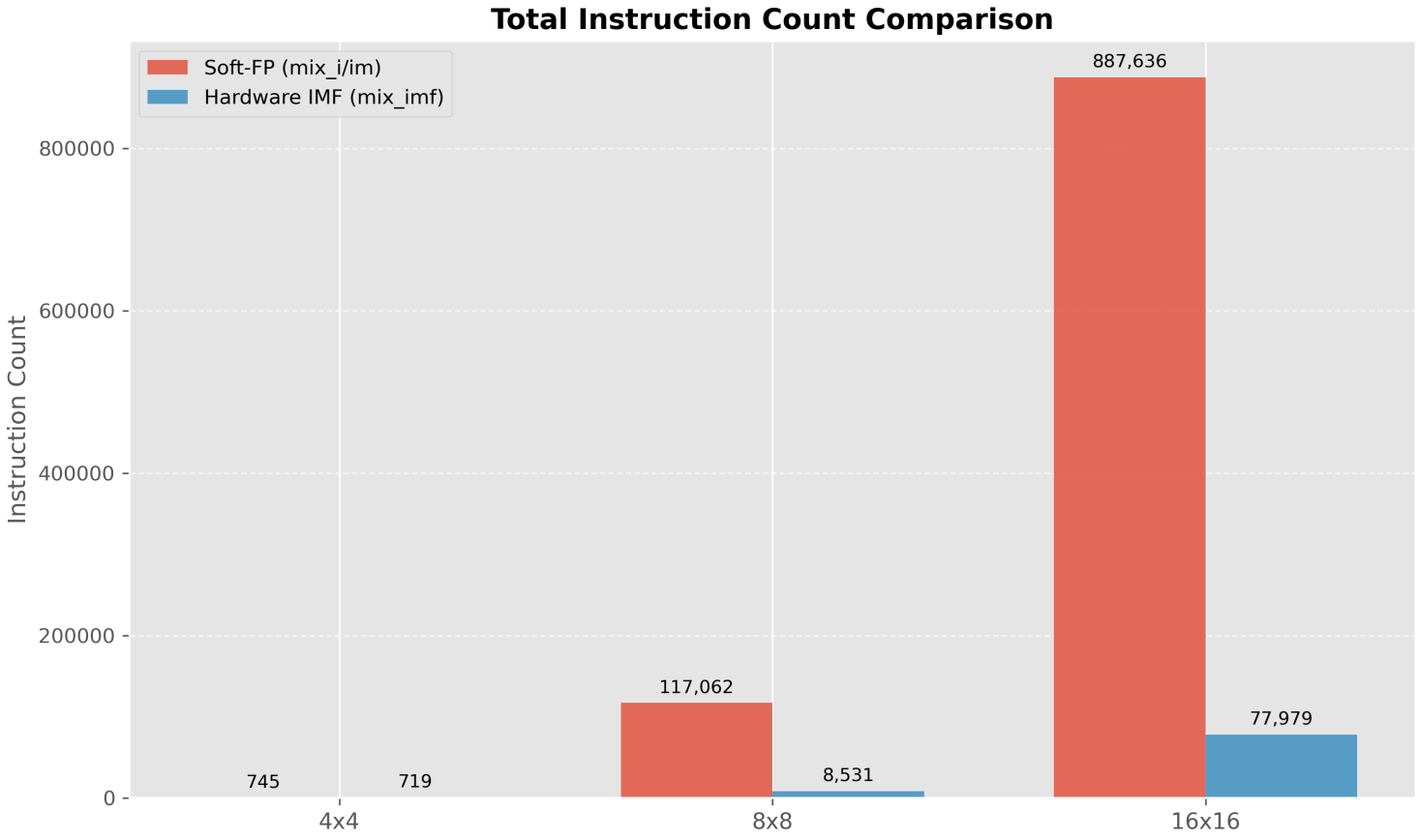


Figure 3: Branch Percentage by Scale